06 - 트랜잭션 풀 및 마이닝 시스템

개요

트랜잭션 풀 및 마이닝 시스템은 Geth에서 트랜잭션의 수집, 관리, 검증 및 블록 생성을 담당하는 핵심 컴포넌트입니다. 이 시스템은 네트워크에서 수신된 트랜잭션들을 효율적으로 관리하고, 마이닝 과정을 통해 새로운 블록을 생성합니다.

주요 구성 요소:

- **TxPool**: 일반 트랜잭션을 관리하는 메인 풀
- **BlobPool**: EIP-4844 블롭 트랜잭션 전용 풀
- **Miner**: 블록 생성 및 마이닝 프로세스 관리
- **Worker**: 실제 블록 생성 작업을 수행하는 워커

핵심 구조체 및 인터페이스

TxPool 구조체

```
// TxPool은 다양한 트랜잭션별 풀들의 집합체로, 노드가 관심을 가지는

// 모든 트랜잭션을 추적합니다.

type TxPool struct {

subpools []SubPool // 전문화된 트랜잭션 처리를 위한 서브풀 목록

chain BlockChain // 블록체인 인터페이스

signer types.Signer // 트랜잭션 서명자
```

```
stateLock sync.RWMutex // 상태 인스턴스 보호를 위한 락
state *state.StateDB // 블록체인 헤드의 현재 상태

subs event.SubscriptionScope // 종료 시 모든 구독을 해제하기 위한 스코프
quit chan chan error // 헤드 업데이터를 종료하기 위한 채널
term chan struct{} // 풀 종료를 감지하기 위한 종료 채널
sync chan chan error // 테스트/시뮬레이터용 동기화 채널
}
```

BlobPool 구조체

```
// BlobPool은 EIP-4844 블롭 트랜잭션을 전문적으로 처리하는 풀입니다.
type BlobPool struct {
config Config // 풀 설정
reserver txpool.Reserver // 서브풀 간 배타성 보장을 위한 주소 예약자
hasPendingAuth func(common.Address) bool // 7702-auth 대기 여부 확인 함수

store billy.Database // 트랜잭션 메타데이터와 블롭을 위한 영구 데이터 저장소
stored uint64 // 디스크상 모든 트랜잭션의 유용한 데이터 크기
limbo *limbo // 확정되지 않은 블롭을 위한 영구 데이터 저장소

signer types.Signer // 발신자 복구를 위한 트랜잭션 서명자
chain BlockChain // 상태 접근을 위한 체인 객체
```

```
head *types.Header // 체인의 현재 헤드
state *state.StateDB // 체인 헤드의 현재 상태
gasTip *uint256.Int // 현재 허용되는 최소 가스 팁

lookup *lookup // 블롭을 트랜잭션에, 트랜잭션을 billy 엔트리에 매핑하는 룩업 테이블
index  map[common.Address][]*blobTxMeta // 계정별로 그룹화되고 논스로 정렬된 블롭 트랜
spent  map[common.Address]*uint256.Int // 개별 계정의 지출 추적
evict *evictHeap // 풀이 가득 찰 때 제거할 가장 저렴한 계정들의 힙

discoverFeed event.Feed // 풀 발견 시 새 트랜잭션 이벤트 전송을 위한 이벤트 피드
insertFeed event.Feed // 풀 포함 시 새 트랜잭션 이벤트 전송을 위한 이벤트 피드

lock sync.RWMutex // 재구성 처리 중 풀 보호를 위한 뮤텍스
}
```

Miner 구조체

```
// Miner는 블록 생성과 마이닝 프로세스를 관리합니다.
type Miner struct {
confMu sync.RWMutex // 설정 변경 보호를 위한 뮤텍스
config *Config // 마이너 설정
chainConfig *params.ChainConfig // 체인 설정
```

```
engine consensus.Engine // 합의 엔진

txpool *txpool.TxPool // 트랜잭션 풀

chain *core.BlockChain // 블록체인

prio []common.Address // 우선순위 계정 목록

}
```

주요 프로세스

1. 트랜잭션 풀 초기화

트랜잭션 풀은 `New()` 함수를 통해 초기화됩니다:

```
// New는 네트워크에서 들어오는 트랜잭션을 수집, 정렬, 필터링하기 위한

// 새로운 트랜잭션 풀을 생성합니다.

func New(gasTip uint64, chain BlockChain, subpools []SubPool) (*TxPool, error)

// 현재 헤드를 가져와서 모든 서브풀과 메인 코디네이터 풀이

// 동일한 시작 상태를 가지도록 합니다.

head := chain.CurrentBlock()


// 헤드 블록으로 상태를 초기화하거나, 헤드 상태를 사용할 수 없는 경우

// (노드가 완전히 동기화되지 않은 경우) 빈 상태로 폴백합니다.

statedb, err := chain.StateAt(head.Root)

if err != nil {

statedb, err = chain.StateAt(types.EmptyRootHash)
```

```go
}
if err != nil {
return nil, err
}
pool := &TxPool{
subpools: subpools,
chain: chain,
signer: types.LatestSigner(chain.Config()),
state: statedb,
quit: make(chan chan error),
term: make(chan struct{}),
sync: make(chan chan error),
}
// 각 서브풀을 초기화합니다.
reserver := NewReservationTracker()
for i, subpool := range subpools {
if err := subpool.Init(gasTip, head, reserver.NewHandle(i)); err != nil {
// 실패 시 이미 초기화된 서브풀들을 정리합니다.
for j := i - 1; j >= 0; j-- {
subpools[j].Close()
}
return nil, err
}
```

```
}
go pool.loop(head) // 메인 이벤트 루프 시작
return pool, nil
}
```

2. 트랜잭션 검증 프로세스

트랜잭션은 풀에 추가되기 전에 여러 단계의 검증을 거칩니다:

기본 검증 (`ValidateTransaction`)

```
// ValidateTransaction은 합의 규칙에 따라 트랜잭션이 유효한지 확인하는
// 헬퍼 메서드입니다. 상태 의존적 검증(잔액, 논스 등)은 확인하지 않습니다.
func ValidateTransaction(tx *types.Transaction, head *types.Header, signer type
// 호출하는 풀에서 구현되지 않은 트랜잭션은 거부합니다.
if opts.Accept&(1<<tx.Type()) == 0 {
return fmt.Errorf("%w: tx type %v not supported by this pool", core.ErrTxTypeNo
}
// 블롭 개수 제한 확인
if blobCount := len(tx.BlobHashes()); blobCount > opts.MaxBlobCount {
return fmt.Errorf("%w: blob count %v, limit %v", ErrTxBlobLimitExceeded, blobCo
}
// 트랜잭션 크기 제한 확인
if tx.Size() > opts.MaxSize {
```

```
return fmt.Errorf("%w: transaction size %v, limit %v", ErrOversizedData, tx.Siz
}
// 활성화된 트랜잭션 타입만 허용
rules := opts.Config.Rules(head.Number, head.Difficulty.Sign() == 0, head.Time)
if !rules.IsBerlin && tx.Type() != types.LegacyTxType {
return fmt.Errorf("%w: type %d rejected, pool not yet in Berlin", core.ErrTxTyp
}
// 트랜잭션 값이 음수가 아닌지 확인
if tx.Value().Sign() < 0 {
return ErrNegativeValue
}
// 현재 블록 가스 한도를 초과하지 않는지 확인
if head.GasLimit < tx.Gas() {
return ErrGasLimit
}
// 가스 수수료 캡이 가스 팁 캡보다 크거나 같은지 확인
if tx.GasFeeCapIntCmp(tx.GasTipCap()) < 0 {
return core.ErrTipAboveFeeCap
}
// 트랜잭션이 올바르게 서명되었는지 확인
if _, err := types.Sender(signer, tx); err != nil {
return fmt.Errorf("%w: %v", ErrInvalidSender, err)
}
```

```
return nil
}
```

상태 기반 검증 (ValidateTransactionWithState)

```
// ValidateTransactionWithState는 상태 의존적 규칙을 확인합니다.
func ValidateTransactionWithState(tx *types.Transaction, signer types.Signer, o
// 트랜잭션이 논스 순서를 준수하는지 확인
from, err := types.Sender(signer, tx)
if err != nil {
return err
}
// 계정의 현재 논스 확인
if nonce := opts.State.GetNonce(from); nonce > tx.Nonce() {
return fmt.Errorf("%w: address %v, tx: %d state: %d", core.ErrNonceTooLow, from
}
// 계정 잔액 확인
balance := opts.State.GetBalance(from)
if balance.Cmp(tx.Cost()) < 0 {
return fmt.Errorf("%w: address %v have %v want %v", core.ErrInsufficientFunds,
}
return nil
```

```
}
```

3. 블록 생성 프로세스

마이닝 시스템의 핵심인 블록 생성은 `generateWork()` 메서드에서 시작됩니다:

```
// generateWork는 주어진 매개변수를 기반으로 봉인 블록을 생성합니다.
func (miner *Miner) generateWork(genParam *generateParams, witness  bool) *newPa
// 작업 환경 준비
work, err := miner.prepareWork(genParam, witness)
if  err != nil {
return &newPayloadResult{err: err}
}

// 출금 요청이 최대 블록 크기에 맞는지 확인
maxBlockSize := params.MaxBlockSize - maxBlockSizeBufferZone
if  genParam.withdrawals.Size() > maxBlockSize {
return &newPayloadResult{err: errors.New("withdrawals exceed max block size")}
}
work.size += uint64(genParam.withdrawals.Size())

// 트랜잭션이 필요한 경우 트랜잭션 채우기
if !genParam.noTxs {
interrupt := new(atomic.Int32)
```

```
timer := time.AfterFunc(miner.config.Recommit, func() {
interrupt.Store(commitInterruptTimeout)
})
defer timer.Stop()

err := miner.fillTransactions(interrupt, work)
if errors.Is(err, errBlockInterruptedByTimeout) {
log.Warn("Block building is interrupted", "allowance", common.PrettyDuration(min
}
}

// 블록 바디 구성
body := types.Body{Transactions: work.txs, Withdrawals: genParam.withdrawals}

// 모든 로그 수집
allLogs := make([]*types.Log, 0)
for _, r := range work.receipts {
allLogs = append(allLogs, r.Logs...)
}

// Prague가 활성화된 경우 합의 레이어 요청 수집
var requests [][]byte
if miner.chainConfig.IsPrague(work.header.Number, work.header.Time) {
```

```
requests = [][]byte{}

// EIP-6110 예치금

if  err := core.ParseDepositLogs(&requests, allLogs, miner.chainConfig);  err !=

return &newPayloadResult{err: err}

}

// EIP-7002 출금 큐 처리

if  err := core.ProcessWithdrawalQueue(&requests, work.evm);  err != nil {

return &newPayloadResult{err: err}

}

// EIP-7251 통합 큐 처리

if  err := core.ProcessConsolidationQueue(&requests, work.evm);  err != nil {

return &newPayloadResult{err: err}

}

}


// 요청 해시 계산 및 설정

if  requests != nil {

reqHash := types.CalcRequestsHash(requests)

work.header.RequestsHash = &reqHash

}


// 합의 엔진을 통한 블록 완료 및 조립

block, err := miner.engine.FinalizeAndAssemble(miner.chain, work.header, work.st
```

```go
if  err != nil {
return &newPayloadResult{err: err}
}


return &newPayloadResult{
block: block,
fees: totalFees(block, work.receipts),
sidecars: work.sidecars,
stateDB: work.state,
receipts: work.receipts,
requests: requests,
witness: work.witness,
}
}
```

4. 트랜잭션 선택 및 포함 프로세스

`fillTransactions()` 메서드는 트랜잭션 풀에서 트랜잭션을 선택하고 블록에 포함시킵니다:

```go
// fillTransactions는 트랜잭션 풀에서 대기 중인 트랜잭션을 가져와서
// 주어진 봉인 블록에 채웁니다.
func (miner *Miner) fillTransactions(interrupt *atomic.Int32, env *environment)
miner.confMu.RLock()
```

```
tip := miner.config.GasPrice

prio := miner.prio

miner.confMu.RUnlock()


// 가스 풀 초기화

if env.gasPool == nil {

env.gasPool = new(core.GasPool).AddGas(env.header.GasLimit)

}


// Prague1 이후: BRIP-0004에 따라 PoL 트랜잭션을 블록에 추가

if miner.chainConfig.IsPrague1(env.header.Number, env.header.Time) {

tx, err := types.NewPoLTx(

miner.chainConfig.ChainID,

miner.chainConfig.Berachain.Prague1.PoLDistributorAddress,

env.header.Number,

params.PoLTxGasLimit,

env.header.BaseFee,

env.header.ParentProposerPubkey,

)

if err != nil {

return fmt.Errorf("failed to create PoL tx: %v", err)

}
```

```
// PoL 트랜잭션 실행
env.state.SetTxContext(tx.Hash(), env.tcount)
if err = miner.commitTransaction(env, tx); err != nil {
log.Trace("Failed to commit PoL transaction", "hash", tx.Hash(), "err", err)
return err
}
}

// 1559/4844 동적 수수료로 사전 필터링된 대기 트랜잭션 검색
filter := txpool.PendingFilter{
MinTip: uint256.MustFromBig(tip),
}
if env.header.BaseFee != nil {
filter.BaseFee = uint256.MustFromBig(env.header.BaseFee)
}
if env.header.ExcessBlobGas != nil {
filter.BlobFee = uint256.MustFromBig(eip4844.CalcBlobFee(miner.chainConfig, env.
}
if miner.chainConfig.IsOsaka(env.header.Number, env.header.Time) {
filter.GasLimitCap = params.MaxTxGas
}

// 일반 트랜잭션과 블롭 트랜잭션을 별도로 가져오기
```

```
filter.OnlyPlainTxs, filter.OnlyBlobTxs = true, false
pendingPlainTxs := miner.txpool.Pending(filter)

filter.OnlyPlainTxs, filter.OnlyBlobTxs = false, true
pendingBlobTxs := miner.txpool.Pending(filter)

// 대기 트랜잭션을 로컬과 원격으로 분할
prioPlainTxs, normalPlainTxs := make(map[common.Address][]*txpool.LazyTransactio
prioBlobTxs, normalBlobTxs := make(map[common.Address][]*txpool.LazyTransaction)

// 우선순위 계정의 트랜잭션 분리
for _, account := range prio {
if txs := normalPlainTxs[account]; len(txs) > 0 {
delete(normalPlainTxs, account)
prioPlainTxs[account] = txs
}
if txs := normalBlobTxs[account]; len(txs) > 0 {
delete(normalBlobTxs, account)
prioBlobTxs[account] = txs
}
}

// 우선순위 트랜잭션 먼저 처리
```

```
if len(prioPlainTxs) > 0 || len(prioBlobTxs) > 0 {
plainTxs := newTransactionsByPriceAndNonce(env.signer, prioPlainTxs, env.header.
blobTxs := newTransactionsByPriceAndNonce(env.signer, prioBlobTxs, env.header.Ba

if err := miner.commitTransactions(env, plainTxs, blobTxs, interrupt); err !=
return err
}
}

// 일반 트랜잭션 처리
if len(normalPlainTxs) > 0 || len(normalBlobTxs) > 0 {
plainTxs := newTransactionsByPriceAndNonce(env.signer, normalPlainTxs, env.heade
blobTxs := newTransactionsByPriceAndNonce(env.signer, normalBlobTxs, env.header.

if err := miner.commitTransactions(env, plainTxs, blobTxs, interrupt); err !=
return err
}
}

return nil
}
```

코드 예시

트랜잭션 풀에 트랜잭션 추가

```
// 트랜잭션 풀 생성
pool, err := txpool.New(gasPrice, blockchain, subpools)
if err != nil {
log.Fatal("Failed to create transaction pool:", err)
}

// 트랜잭션 추가
tx := types.NewTransaction(nonce, to, value, gasLimit, gasPrice, data)
signedTx, err := types.SignTx(tx, signer, privateKey)
if err != nil {
log.Fatal("Failed to sign transaction:", err)
}

// 풀에 트랜잭션 추가
errs := pool.Add([]*types.Transaction{signedTx}, true, false)
for _, err := range errs {
if err != nil {
log.Printf("Failed to add transaction: %v", err)
}
```

```
}
```

블롭 풀 사용 예시

```
// 블롭 풀 생성
blobPool := blobpool.New(config, blockchain, hasPendingAuth)

// 블롭 풀 초기화
err := blobPool.Init(gasPrice, head, reserver)
if err != nil {
log.Fatal("Failed to initialize blob pool:", err)
}

// 블롭 트랜잭션 필터링
if blobPool.Filter(tx) {
// 블롭 트랜잭션인 경우 블롭 풀에서 처리
errs := blobPool.Add([]*types.Transaction{tx}, true, false)
for _, err := range errs {
if err != nil {
log.Printf("Failed to add blob transaction: %v", err)
}
}
}
```

마이너 설정 및 블록 생성

```
// 마이너 생성
miner := miner.New(eth, &config, chainConfig, mux, engine, isLocalBlock)

// 마이닝 시작
miner.Start()

// 블록 생성 매개변수 설정
genParams := &generateParams{
timestamp: uint64(time.Now().Unix()),
forceTime: false,
parentHash: common.Hash{},
coinbase: coinbaseAddr,
withdrawals: withdrawals,
noTxs: false,
}

// 블록 생성
result := miner.generateWork(genParams, false)
if  result.err != nil {
log.Printf("Failed to generate block: %v", result.err)
} else {
```

```
log.Printf("Generated block: %s", result.block.Hash().Hex())

}
```

최적화 및 성능

1. 트랜잭션 정렬 최적화

트랜잭션은 가격과 논스에 따라 정렬되어 최적의 수수료 수익을 보장합니다:

```
// newTransactionsByPriceAndNonce는 가격과 논스에 따라 트랜잭션을 정렬합니다.

func newTransactionsByPriceAndNonce(signer types.Signer, txs map[common.Addres

// 각 계정의 트랜잭션을 논스 순으로 정렬

for _, accTxs := range txs {

sort.Slice(accTxs, func(i, j int) bool {

return accTxs[i].Tx.Nonce() < accTxs[j].Tx.Nonce()

})

}

// 가격 기준 힙 생성

heads := make(txByPriceAndTime, 0, len(txs))

for from, accTxs := range txs {

if len(accTxs) > 0 {

heads = append(heads, &txWithMinerFee{

tx: accTxs[0],

from: from,
```

```
fees: accTxs[0].Tx.EffectiveGasTip(baseFee),
feePerGas: accTxs[0].Tx.GasFeeCap(),
})
}
}
heap.Init(&heads)
return &transactionsByPriceAndNonce{
txs: txs,
heads: heads,
signer: signer,
baseFee: baseFee,
}
}
```

2. 메모리 관리

블롭 풀은 디스크 기반 저장소를 사용하여 메모리 사용량을 최적화합니다:

```
// BlobPool의 메모리 효율적인 저장
type BlobPool struct {
store billy.Database // 디스크 기반 영구 저장소
stored  uint64  // 저장된 데이터 크기 추적
limbo *limbo // 확정되지 않은 블롭을 위한 별도 저장소
// 메모리 내 인덱스만 유지
```

```
lookup *lookup // 빠른 검색을 위한 룩업 테이블
index  map[common.Address][]*blobTxMeta // 계정별 트랜잭션 메타데이터
evict *evictHeap // 제거 우선순위 힙
}
```

3. 동시성 처리

트랜잭션 풀은 읽기-쓰기 락을 사용하여 동시성을 최적화합니다:

```
// 상태 보호를 위한 읽기-쓰기 락 사용
type TxPool struct {
stateLock sync.RWMutex // 상태 보호
state *state.StateDB // 현재 상태
}

// 읽기 작업
func (p *TxPool) Stats() (int, int) {
p.stateLock.RLock()
defer  p.stateLock.RUnlock()
// 상태 읽기 작업
}

// 쓰기 작업
func (p *TxPool) reset(oldHead, newHead *types.Header) {
```

```
p.stateLock.Lock()
defer  p.stateLock.Unlock()
// 상태 업데이트 작업
}
```

디버깅 및 모니터링

1. 로깅 포인트

주요 로깅 포인트들:

```
// 트랜잭션 추가 시 로깅
log.Debug("Transaction added to pool", "hash", tx.Hash(), "from", from, "to", tx

// 블록 생성 시 로깅
log.Info("Generated new block", "number", block.Number(), "hash", block.Hash(),
"txs", len(block.Transactions()), "gas", block.GasUsed())

// 트랜잭션 제거 시 로깅
log.Trace("Transaction evicted from pool", "hash", tx.Hash(), "reason", reason)

// 마이닝 중단 시 로깅
log.Warn("Block building is interrupted", "allowance", common.PrettyDuration(tim
```

2. 메트릭 수집

성능 모니터링을 위한 메트릭들:

```
// 트랜잭션 풀 메트릭
var (
pendingGauge = metrics.NewRegisteredGauge("txpool/pending", nil)
queuedGauge = metrics.NewRegisteredGauge("txpool/queued", nil)
localGauge = metrics.NewRegisteredGauge("txpool/local", nil)
knownTxMeter = metrics.NewRegisteredMeter("txpool/known", nil)
validTxMeter = metrics.NewRegisteredMeter("txpool/valid", nil)
invalidTxMeter = metrics.NewRegisteredMeter("txpool/invalid", nil)
)

// 블록 생성 메트릭
var (
blockGenerationTimer = metrics.NewRegisteredTimer("miner/block/generation", nil)
txCommitTimer = metrics.NewRegisteredTimer("miner/tx/commit", nil)
blockSizeGauge = metrics.NewRegisteredGauge("miner/block/size", nil)
)
```

3. 상태 검사 함수

시스템 상태를 확인하기 위한 함수들:

```go
// 트랜잭션 풀 상태 확인
func (p *TxPool) Stats() (int, int) {
p.stateLock.RLock()
defer p.stateLock.RUnlock()
pending, queued := 0, 0
for _, subpool := range p.subpools {
p, q := subpool.Stats()
pending += p
queued += q
}
return pending, queued
}

// 블롭 풀 상태 확인
func (p *BlobPool) Stats() (int, int) {
p.lock.RLock()
defer p.lock.RUnlock()
return len(p.lookup.slots), 0 // 블롭 풀은 큐가 없음
}

// 마이너 상태 확인
func (miner *Miner) Mining() bool {
```

```
return  atomic.LoadInt32(&miner.shouldStart) == 1 && miner.worker.isRunning()
}
```

관련 컴포넌트

의존성 관계

1. **BlockChain**: 현재 상태와 헤드 블록 정보 제공
2. **StateDB**: 계정 잔액과 논스 정보 제공
3. **Consensus Engine**: 블록 완료 및 검증 수행
4. **EVM**: 트랜잭션 실행 환경 제공
5. **P2P Network**: 트랜잭션 전파 및 수신

상호작용 방식

```
// 트랜잭션 풀 → 블록체인
currentBlock := pool.chain.CurrentBlock()
state, err := pool.chain.StateAt(currentBlock.Root)

// 마이너 → 트랜잭션 풀
pendingTxs := miner.txpool.Pending(filter)

// 마이너 → 합의 엔진
block, err := miner.engine.FinalizeAndAssemble(chain, header, state, body, recei
```

```
// 트랜잭션 풀 → EVM (간접적으로 상태 변경을 통해)

receipt, err := core.ApplyTransaction(chainConfig, chain, &coinbase, gp, statedb
```

트랜잭션 풀 및 마이닝 시스템은 Geth의 핵심 기능 중 하나로, 네트워크 트랜잭션의 효율적인 관리와 블록 생성을 통해 이더리움 네트워크의 안정적인 운영을 보장합니다.